

**CUỘC THI KHOA HỌC, KỸ THUẬT DÀNH CHO HỌC SINH TRUNG HỌC
CẤP TRƯỜNG NĂM HỌC 2026 – 2027**

BẢN BÁO CÁO TÓM TẮT KẾT QUẢ NGHIÊN CỨU
(Report on the Research's Findings)

Tên đề tài:

**MEMO-LANG: NGHIÊN CỨU VÀ CHẾ TẠO ỨNG DỤNG CƠ CHẾ
GIẢI PHÓNG DỮ LIỆU ĐỆM TRONG PHÁT TRIỂN AI**

Lĩnh vực: Phần mềm hệ thống

Hà Nội, tháng 9 năm 2026

MỤC LỤC

Tổng quan dự án	3
PHẦN I: Novelty	3
PHẦN II: Background	4
PHẦN III: Project Goal	5
III.1. Destructive-Read Semantics	6
PHẦN IV: Compiler and Runtime Architecture	6
PHẦN V: Maths and Algorithms	6
V.1. Math Behind FP32 Matrix Multiplication	6
V.2. The Free-on-Consume Scheme	6
PHẦN VI: CUTLASS Integration	6
PHẦN VII: Verification	7
PHẦN VIII: Benchmarks	7
VIII.1. Expected Relationship Between Language and MatMul Performance	8
PHẦN IX: Evidence and Reproducibility	8
PHẦN X: Conclusion	8
References	8

DANH MỤC HÌNH VẼ, SƠ ĐỒ

DANH MỤC BẢNG BIỂU

Bảng 1. Main Source Files	6
Bảng 2. Correctness Test Results (Template)	7

Sự phát triển của AI trong thời gian gần đây đã dẫn đến sự bùng nổ của nhiều thuật toán máy học và mô hình học sâu. Do vậy, nhu cầu cho năng lực tính toán để học trên những bộ dữ liệu khổng lồ cũng từ đó tăng.

Hiện nay, phép toán được sử dụng rộng rãi cho thuật toán máy học là GEMM (General Matrix Multiplication - Phép nhân ma trận tổng quát). Trong quá trình học, mỗi phép nhân ma trận sẽ tạo ra một ma trận khác và ma trận mới sẽ được nối với những phép nhân ma trận khác. Để xử lý nhiều thao tác, tác vụ và quá trình khác nhau, những mô hình máy học cần thực hiện hàng triệu phép tính ma trận. Do những phép tính ma trận này so với các phép tính thông thường khác có tính đặc thù về thời gian rất cao, vì vậy những phép tính tiếp theo dần bị dồn lên bộ nhớ đệm và các bước trung gian cho đến khi những chương trình quản lý bộ nhớ hoặc quy tắc của máy tính giải phóng bộ nhớ.

Việc lặp lại quá trình này đối với mô hình máy học trở thành một vấn đề lớn trong việc hoạt động dưới một khối lượng công việc rất lớn, để lại hậu quả lớn cho hiệu suất và độ trễ của mô hình trên thiết bị và tài nguyên phần cứng nó đang sử dụng.

Bởi vậy, phần mềm đồng thời góp phần tối ưu hóa nhân của GEMM và tăng tuổi thọ của phần cứng được sử dụng, đảm bảo bộ đệm, dữ liệu đệm và con trỏ được giải phóng sớm nhất có thể và giảm phí tổn từ kết quả của việc dồn bộ nhớ đệm.

PHẦN I: Novelty

Our main goal is to directly link the precompiled cuBLAS runtime library for a specific application, evaluated with a specific methodology, under an enforcement model resembling a strictly linear type system. Other languages have pursued related consumption-tracking semantics: Rust’s ownership system [6] enforces a comparable use-once discipline through its affine type system, and Futhark [7] is a purely functional array language that uses uniqueness types to support safe in-place updates. Futhark, however, generates its own GPU kernels from source rather than integrating with external, independently maintained high-performance libraries such as cuBLAS or CUTLASS – a limitation that has been noted as one of the practical hurdles of the language. MemoLang instead separates compile-time consumption analysis from kernel execution, delegating the arithmetic itself to an externally linked, production-grade GEMM library.

$$T_1 = AB, \quad T_2 = T_1C, \quad T_3 = T_2D, \quad (1)$$

In a conventional implementation, several intermediate buffers may remain live at the same time. If A and T_1 are no longer required after their respective operations, destructive-read semantics may allow their storage to be reused.

The expected benefit is therefore primarily a memory-management benefit, rather than an improvement to the arithmetic throughput of an individual matrix multiplication. If MemoLang generates the same optimized CUTLASS or cuBLAS kernel as an equivalent implementation, the GPU kernel itself should have approximately the same computational performance. Any end-to-end improvement would instead result from factors

such as reduced temporary allocation, reduced buffer lifetime, or reduced memory pressure.

This distinction is important because an optimized GPU matrix multiplication can dominate the execution time of a large workload. In such cases, language-level optimization may have little effect on kernel execution time. Conversely, workloads containing many intermediate tensors or relatively small matrix operations may spend a greater fraction of their execution time on memory management, making them more suitable for evaluating destructive reads.

The project therefore investigates the following hypotheses:

- Destructive-read semantics can reduce peak memory usage for workloads containing short-lived intermediate matrices.
- Destructive reads can reduce the number or cost of temporary-buffer allocations when consumed buffers can safely be reused.
- For workloads in which memory management represents a significant portion of total execution time, buffer reuse may improve end-to-end performance.

This project was inspired by research into destructive-read memory systems, the Sere programming language [5], the Clean programming language [4], and optimized GPU matrix-multiplication libraries.

Potential real-world benefits include:

- Improved performance for matrix multiplication operations, supporting faster training and inference for machine learning models.
- A lower hardware-specification entry barrier for developers implementing machine learning algorithms and LLM-related code, by providing a high-level language that abstracts away low-level optimization details.
- More effective compiler-driven reuse of intermediate tensor buffers.
- Reduced run-time buffer and cache allocation overhead.

PHẦN II: Background

Matrix multiplication can be written as

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j} \quad (2)$$

Each value in the output matrix C is produced by multiplying one row of A with one column of B and summing the products. Efficient implementations must manage data movement, memory reuse, and, in a destructive-read scheme, the lifetime of operand buffers once they are consumed.

Destructive-read is a memory read operation that inherently erases the value it reads: data is destroyed at the same time it is accessed. If a value must be preserved, a write-back operation must immediately restore it, or an external buffer/cache must hold a copy.

In modern software, this practice is rarely used directly, owing to its implementation complexity. Contemporary programming languages instead use ownership and immutability to implement move semantics, aiming for similar safety and memory-reuse guarantees without the sharp edges of unsynchronized destructive reads. Related work on linear type systems takes a similar approach at the language level, requiring that a variable be used exactly once.

Destructive-read has, historically, been more common in hardware and circuit design than in software. Dybdahl et al. [1] studied destructive-read behavior in embedded DRAM and found that removing the write-back buffer and instead relying on the cache – reading data into the cache without immediately restoring it to DRAM, and writing it back only when the cache line is replaced – offered measurable power and performance benefits. While that result is specific to a hardware memory system, it motivates the broader question this project investigates: whether a comparable consume-on-read discipline, applied at the software/interpreter level for GEMM-heavy workloads, can yield analogous benefits in memory pressure and allocation overhead.

PHẦN III: Project Goal

The goal of MemoLang is to determine whether language-level destructive-read semantics can provide measurable memory-management benefits for matrix-oriented GPU programs while retaining the performance of optimized matrix multiplication kernels.

The project will be implemented and evaluated through the following goals:

- Define MemoLang’s core Python-like syntax and destructive-read semantics.
- Implement a lexer, parser, AST, and LLVM-based code-generation pipeline.
- Generate LLVM IR for arithmetic and matrix operations.
- Integrate optimized GPU matrix multiplication through CUTLASS and/or cuBLAS.
- Implement explicit ownership and consumption tracking for matrix operands.
- Determine when consumed matrix buffers can safely be reused.
- Verify generated results against an independent numerical reference.
- Measure peak GPU memory usage and temporary-buffer allocation behavior.
- Measure both GPU kernel execution time and end-to-end program runtime.
- Compare destructive-read execution against an equivalent non-destructive implementation.
- Compare MemoLang against optimized GPU-library baselines where appropriate.
- Preserve benchmark logs, correctness results, compiler output, and build artifacts to make the experimental results reproducible.

The evaluation will not assume that destructive reads provide a performance improvement. A reduction in peak memory usage without a reduction in runtime will be treated as a meaningful result, while an observed runtime improvement will be analyzed to determine whether it originates from reduced memory-management overhead rather than from changes to the underlying matrix multiplication kernel.

III.1. Destructive-Read Semantics

A destructive read treats a value as consumed when an operation has exclusive use of that value. After consumption, the original value is no longer available to subsequent operations unless the programmer explicitly creates a copy or otherwise preserves it.

For matrix operations, this provides an opportunity for storage reuse. For example,

$$\begin{aligned}C &= A @ B \\ D &= C @ W\end{aligned}$$

may permit the storage associated with A or C to become eligible for reuse once the compiler establishes that the corresponding value has no remaining consumers.

The purpose of this mechanism is not to alter the mathematical computation performed by matrix multiplication. Instead, it changes the lifetime of buffers associated with the computation. If a buffer can be reused without copying its contents, the implementation may reduce peak memory consumption and allocation activity.

This optimization is inherently workload-dependent. A matrix with multiple future consumers cannot safely be destroyed after its first use. In such cases, the implementation may require a copy, borrow, or other mechanism to preserve the value. Consequently, destructive-read semantics can introduce a tradeoff between memory efficiency and the flexibility to reuse values.

The language therefore treats destructive reads as an explicit ownership mechanism rather than assuming that consuming every operand will necessarily improve performance.

PHẦN IV: Compiler and Runtime Architecture

PHẦN V: Maths and Algorithms

V.1. Math Behind FP32 Matrix Multiplication

V.2. The Free-on-Consume Scheme

PHẦN VI: CUTLASS Integration

Bảng 1. Main Source Files

File	Purpose
Lexer.h/.cpp	Tokenizes MemoLang source
Parser.h/.cpp	Builds the AST
ExprAST.h	AST node definitions
CodeGen.cpp	LLVM IR generation
CuBLASRuntime.cpp	cuBLAS MatMul bindings
main.cpp	Driver / REPL

Bảng 2. Correctness Test Results (Template)

Test Case	Result
$N \times N$ MatMul vs. reference	TBD
Destructive-read re-use error detection	TBD
Edge cases (empty, 1x1, non-square)	TBD

PHẦN VII: Verification**PHẦN VIII: Benchmarks**

The benchmark evaluation is designed to distinguish improvements in memory management from improvements in matrix multiplication itself.

Three primary implementations will be compared:

1. An optimized GPU-library implementation used as a kernel-performance reference.
2. MemoLang using conventional non-destructive value semantics.
3. MemoLang using destructive-read semantics.

Where appropriate, a simple C++ or CUDA implementation will also be included as a baseline.

The most important comparison is between the two MemoLang configurations. Both configurations perform the same mathematical operations and use the same underlying GPU matrix multiplication implementation. This comparison isolates the effect of destructive-read semantics more effectively than a comparison against an unrelated implementation.

The benchmark suite will include isolated matrix multiplications as well as chains of matrix operations. Matrix dimensions will include small, medium, large, and non-square matrices. This allows the evaluation to determine whether the effect of destructive reads depends on workload size.

The following measurements will be collected:

- GPU kernel execution time.
- End-to-end execution time.
- Peak GPU memory consumption.
- Number of temporary-buffer allocations.
- Number of buffer reuse events.
- Host-to-device and device-to-host transfer time where applicable.
- Compiler and runtime overhead.
- Numerical error relative to a reference implementation.

GPU measurements will use multiple repetitions and appropriate warm-up iterations. CUDA initialization and compilation overhead will be separated from steady-state execution measurements where possible.

The benchmark will report both absolute measurements and relative changes. No performance improvement will be attributed to destructive-read semantics unless the corresponding measurements support that interpretation.

VIII.1. Expected Relationship Between Language and MatMul Performance

The expected execution time can be approximately decomposed as

$$T_{\text{total}} = T_{\text{language}} + T_{\text{memory}} + T_{\text{launch}} + T_{\text{kernel}}. \quad (3)$$

Destructive-read semantics primarily target T_{memory} . They are not expected to substantially reduce T_{kernel} when the same optimized GPU kernel is used.

For a large matrix multiplication, T_{kernel} may dominate the total execution time, resulting in little observable end-to-end improvement. For workloads containing many intermediate tensors or small matrix operations, however, T_{memory} and other runtime costs may become more significant.

Consequently, a successful result does not require MemoLang to make an individual matrix multiplication faster. A reduction in peak memory usage, allocation activity, or end-to-end overhead can constitute a meaningful result even when kernel execution time remains approximately unchanged.

PHẦN IX: Evidence and Reproducibility

PHẦN X: Conclusion

References

- [1] Haakon Dybdahl, Per Gunnar Kjeldsberg, Marius Grannæs, and Lasse Natvig, “Destructive-read in embedded DRAM, impact on power consumption,” *Journal of Embedded Computing*, vol. 2, no. 1, pp. 83–96, 2006.
- [2] NVIDIA, “cuBLAS Library.” <https://developer.nvidia.com/cublas>
- [3] LLVM Project, “The LLVM Compiler Infrastructure.” <https://llvm.org>
- [4] Clean Language Team, “The Clean Programming Language.” <https://clean.cs.ru.nl>
- [5] Sere Language Project, “Sere Programming Language.” <https://clean-lang.org>
- [6] N. D. Matsakis and F. S. Klock, “The Rust language,” in *Proc. ACM SIGAda Ada Letters*, 2014. <https://www.rust-lang.org>
- [7] T. Henriksen, N. G. W. Serup, M. Elsmann, F. Henglein, and C. E. Oancea, “Futhark: purely functional GPU-programming with nested parallelism and in-place array up-

dates,” in *Proc. 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017. <https://futhark-lang.org>

- [8] Independent Interpreter Project, “MemoLang: source and benchmark evidence,” project repository, September 2026.